

Asynchronous Message Delivery – documentation du webservice REST

Table of Contents

1. Versions.....	1
2. Contexte	2
2.1. e-Gov 3.0.....	2
2.2. Projet LTDS.....	2
2.3. Projet AMD	2
2.4. AMD en combinaison avec LTDS.....	2
3. Webservice AMD (API RESTful).....	2
3.1. Qu'est-ce que AMD.....	2
3.2. Groupes d'utilisateurs	3
3.3. Environnements et URL.....	3
3.4. Aperçu des endpoints API	3
3.4.1. POST /messages/consume.....	4
3.4.2. POST /messages/resetIterator	6
3.5. Le message	6
3.5.1 Type d'événement CloudEvents	6
3.5.2 Unicité d'un message	7
3.5.3 Durée de conservation.....	7
4. Accès à AMD.....	7
4.1 Configuration Chaman.....	7
4.2 Token d'accès via OAuth	7
5. Contact.....	8

1. Versions

Version	Date	Description
0.1	2026-03-17	Publication d'une version pilote du service <i>Asynchronous Message Delivery</i> prévue pour juin 2026

2. Contexte

2.1. e-Gov 3.0

e-Gov 3.0 est un programme de la sécurité sociale belge (ONSS) visant à rendre les services publics numériques plus rapides, plus simples et plus efficaces.

L'un des objectifs est de moderniser la sécurité sociale, notamment en construisant une structure numérique pérenne. Cela passe par exemple par le remplacement de la déclaration trimestrielle DmfA par des transferts de données salariales rationalisés.

Plus d'informations sur e-Gov 3.0 : <https://www.egov.securitesociale.be/fr/>

2.2. Projet LTDS

LTDS (Loondata-Transfert / Transfert de Données Salariales) est un nouveau projet de service en ligne de la ONSS, faisant partie du programme e-Gov 3.0.

Il vise à moderniser la déclaration trimestrielle DmfA actuelle et, à terme, à la remplacer par un échange structuré de données salariales facilitant la communication des informations de calcul salarial des employeurs et mandataires vers la ONSS.

Plus d'informations sur le projet LTDS : <https://github.com/smals-belgium/egov3-services-catalog?tab=readme-ov-file>

2.3. Projet AMD

Afin de répondre aux objectifs de e-Gov 3.0, il est nécessaire, en plus d'une saisie plus rapide des données via le système LTDS, d'envoyer plus rapidement le feedback des données traitées aux utilisateurs.

Le service **AMD (Asynchronous Message Delivery)** a pour objectif de rendre les données et autres formes de feedback disponibles de manière efficace et en temps utile pour l'utilisateur.

2.4. AMD en combinaison avec LTDS

Les messages de feedback relatifs aux données introduites via le webservice *salaryData* du LTDS seront exclusivement renvoyés via le service AMD.

Les données transmises via un canal batch ou via une application web du portail de la sécurité sociale ne recevront pas de feedback via AMD.

3. Webservice AMD (API RESTful)

3.1. Qu'est-ce que AMD

Le webservice AMD permet de rendre des messages disponibles de manière asynchrone aux utilisateurs. Il agit comme intermédiaire entre un service producteur de messages (le

producteur) et un service qui les traite (le consommateur), les découplant ainsi l'un de l'autre. Cela permet un flux continu de messages que le consommateur peut traiter à son propre rythme.

Actuellement, le webservice AMD est uniquement lié au webservice *salaryData* et ne disposera de messages pour une entreprise que si les données correspondantes ont été introduites via ce webservice.

3.2. Groupes d'utilisateurs

En tant que « consommateur », le webservice AMD peut être utilisé par les systèmes IT de :

- secrétariats sociaux agréés (SSA)
- prestataires de services
- employeurs

En tant que consommateur, il est uniquement possible de récupérer les messages qui vous sont destinés, en tant qu'entreprise/rôle d'entité (cette combinaison correspond à un numéro expéditeur unique dans le système Chaman, voir aussi la section 4).

3.3. Environnements et URL

- simulation: <https://services-sim.socialsecurity.be/REST/asynchronousMessageDelivery/v1>
- production: <https://services.socialsecurity.be/REST/asynchronousMessageDelivery/v1>

3.4. Aperçu des endpoints API

La spécification de l'API est disponible via :

<https://smals-belgium.github.io/egov3-services-catalog/?spec=amed>

Elle comprend les endpoints suivants :

- **Pour le producteur (scope `scope:rsz-onss:asynchronous:message-delivery-rest:producer`)**
 - POST /messageDeliveries/produce
- **Pour le consommateur (scope `scope:rsz-onss:asynchronous:message-delivery-rest:consumer`)**
 - POST /messages/consume
 - POST /messages/resetIterator

Lorsque nous parlons d'utilisateurs consommant des messages, nous faisons référence à des services externes agissant uniquement comme consommateurs.

Les webservices internes de l'ONSS agissent comme producteurs de messages et sont responsables de leur création et de leur mise à disposition.

Les services externes ne peuvent utiliser les endpoints suivants que s'ils y sont autorisés :

- POST /messages/consume pour récupérer les messages disponibles
- POST /messages/resetIterator pour ajuster l'itérateur

Voir la section **4. Accès à AMD** pour plus d'informations.

3.4.1. POST /messages/consume

Un utilisateur autorisé (consommateur) peut recevoir les derniers messages disponibles via cet endpoint.

Grâce au token d'autorisation, le numéro d'entreprise et le numéro d'envoi sont récupérés, permettant à AMD d'identifier l'utilisateur.

Les messages disponibles sont ensuite sélectionnés et envoyés au service consommateur.

Plus d'informations sur le concept d'expéditeur et de numéro d'envoi :

https://www.socialsecurity.be/site_fr/employer/applics/chaman/general/faq.htm

Les messages sont envoyés par lots de maximum 100 messages par appel.

Un paramètre **iterator** est toujours fourni pour la pagination.

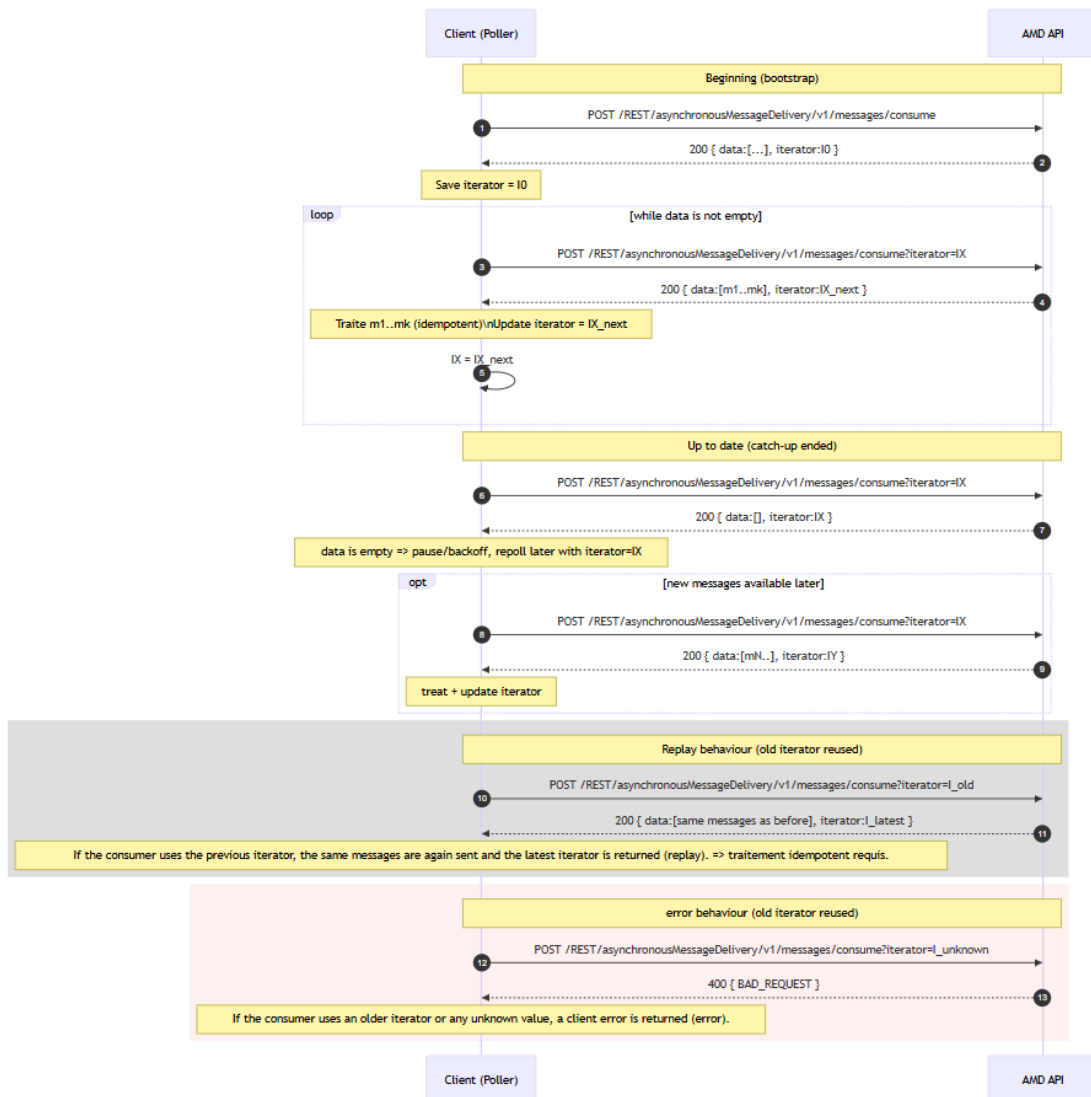
- Il indique la position actuelle du dernier message consommé
- Il est utilisé pour récupérer le lot suivant

Si nécessaire, l'itérateur actuel peut être récupéré en envoyant une requête avec un paramètre iterator vide.

Lorsque tous les messages ont été consommés, une liste vide est renvoyée avec l'itérateur actuel.

Valeurs possibles de l'iterator :

- **Iterator vide** : renvoie le dernier iterator et la dernière liste consommée (ou première liste si aucun message consommé)
- **Iterator actuel** : renvoie l'iterator suivant et la liste suivante
- **Iterator précédent** : renvoie l'iterator suivant et la dernière liste consommée
- **Toute autre valeur** : erreur



3.4.1.1 Intervalle de polling

Pour éviter de surcharger le service AMD, il est recommandé d'utiliser un intervalle de polling raisonnable :

- Tant qu'il reste des messages : polling continu possible
- Une fois terminé : vérifier toutes les 30 à 60 secondes maximum

3.4.1.2 Statuts des messages

- **CONSUMED** : message récupéré par le consommateur
- **ACKNOWLEDGED** : message confirmé après récupération du lot suivant

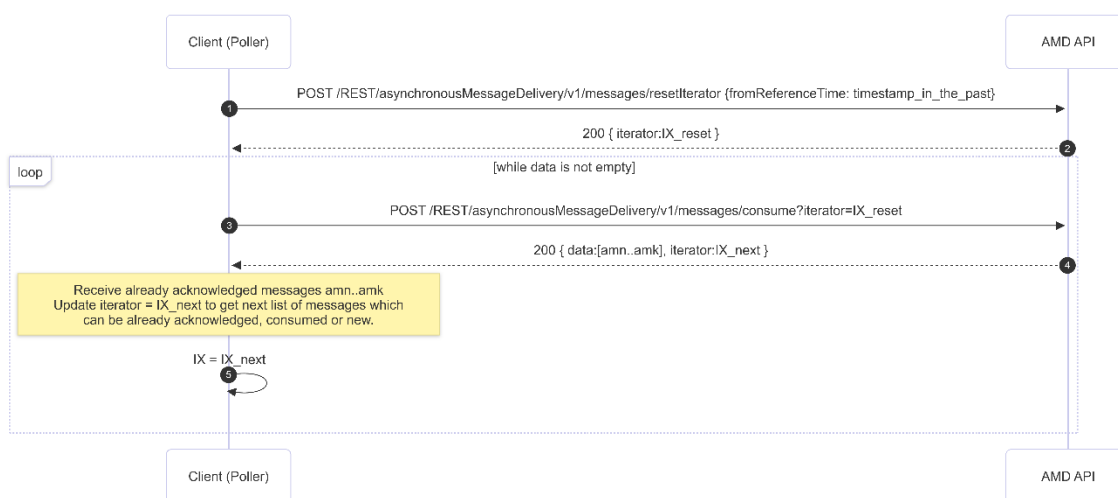
Cela permet au producteur de savoir que le message a été reçu.

3.4.2. POST /messages/resetIterator

Cette opération permet de repositionner l'iterator à un moment donné afin de récupérer à nouveau des messages déjà consommés.

Contraintes :

- Impossible de dépasser le moment du dernier message confirmé (avec le statut ACKNOWLEDGED)
- Les messages sont conservés 6 mois
- Un iterator trop ancien renverra uniquement les premiers messages encore disponibles



Attention :

Après un `resetIterator`, des requêtes utilisant un ancien iterator peuvent générer des erreurs. Pour récupérer le dernier iterator, envoyer une requête avec iterator vide.

En cas de messages ayant exactement le même timestamp, AMD positionnera l'iterator sur le premier message correspondant.

3.5. Le message

Les événements sont transmis sous forme de messages.

AMD agit uniquement comme intermédiaire et ne connaît pas le contenu du message.

3.5.1 Type d'événement CloudEvents

Les messages utilisent le standard **CloudEvents**, qui uniformise les métadonnées (id, source, type).

Le message reçu par le consommateur est identique à celui envoyé par le producteur.

Spécification CloudEvents :

<https://github.com/cloudevents/spec/blob/main/cloudevents/spec.md>

Pour cette version pilote, la source des messages est l'équipe LTDS (LTDS@smals.be).

3.5.2 Unicité d'un message

L'unicité est garantie par :

- **id** : identifiant unique dans le contexte de la source
- **source** : origine de l'événement

La combinaison des deux doit être unique.

Cela permet d'éviter les doublons (modèle *at-least-once delivery*).

3.5.3 Durée de conservation

Les messages sont conservés pendant **6 mois**, puis supprimés, qu'ils aient été consommés ou non.

4. Accès à AMD

Les systèmes IT communiquent avec l'API AMD via les canaux sécurisés du portail de la Sécurité Sociale.

4.1 Configuration Chaman

L'entreprise doit être configurée comme expéditeur pour AMD dans Chaman.

Documentation :

- https://www.socialsecurity.be/site_fr/employer/applics/chaman/index.htm
- <https://www.rest-documentation.socialsecurity.be/documentation/activating-and-managing-your-rest-channel-in-chaman.html>

Les webservices suivants doivent être activés :

- Loondata-Transfert
- AsynchronousMessageDelivery

4.2 Jeton d'accès (access token) via OAuth

Toutes les requêtes nécessitent un token OAuth 2.0 valide, émis par l'Authorization Server de la Sécurité Sociale.

1. Demander un access token

Envoyer une requête au serveur OAuth de la sécurité sociale avec les identifiants client

(client ID et secret). Documentation :

<https://www.rest-documentation.socialsecurity.be/documentation/developer-guide/obtaining-an-access-token-from-social-security-oauth-server.html>

Pour l'utilisation du service AMD, vous devez alors obtenir un access token qui inclut le scope `scope:rsz-onss:asynchronous:message-delivery-rest:consumer`

2. **Utiliser le token dans les requêtes API**

Inclure le token dans le header Authorization.

5. Contact

Pour toute question, contacter l'équipe AMD :

EgovNoti-team@smals.be